

lab1 Socket实现多人聊天室

才子桓 2212700 密码科学与技术

实验说明

本实验利用 c++ 语言，使用流式 Socket 完成一个多人聊天室的程序。该实验成果分为服务器端（Server）和客户端（Client）。在本机 IP（127.0.0.1）下，选择一个空闲的端口号，Server 端开启监听，可供连接该服务器的客户端进行实时聊天，发送包括数字、英文、中文以及符号的信息，可以通过输入 `exit` 退出聊天室。本实验未提供图形化界面，使用终端命令行进行通信。

实验环境

开发IDE： Visual Studio 2022

语言： c++14

注：在解决方案属性中关闭 SDL 检查

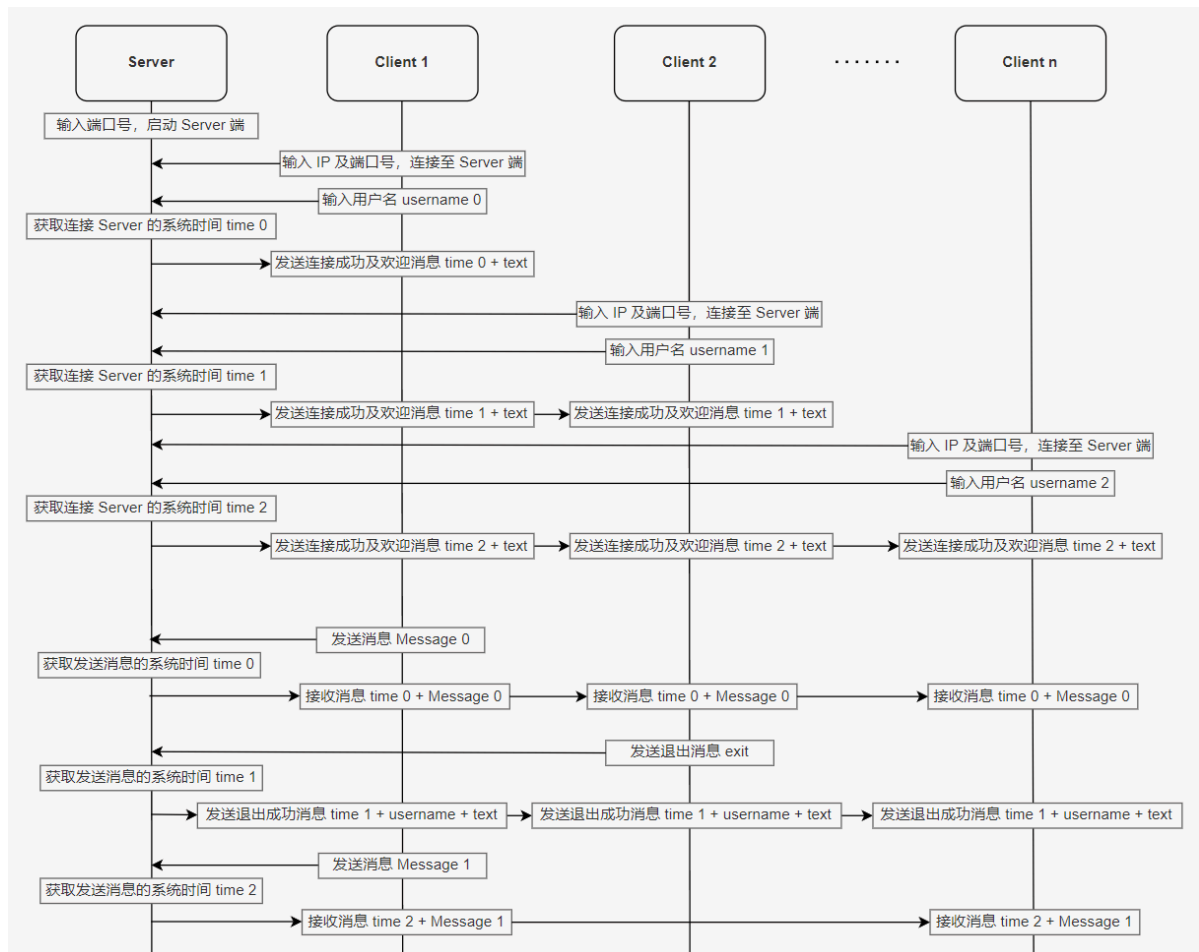
实现思路

协议实现

语法及语义： 在本次实验一共设计了以下3种消息类型：

1. 在 Client 首次连接服务器端时，需要发送自己的用户名。即 `[username]`，同时 Server 端会获取本机当前时间，组成消息 `[time]+[username]`;
2. 在通讯过程，Client 端发送消息，包括消息发送目标 `[target]` 和消息 `[message]`，因为本实现实现的是多人聊天室，发送目标默认设置为连接该 Server 的所有 Client，即发送 `[target]+[message]`;
3. 如要结束通讯退出聊天室，本实验提供一种正常的退出方式 `exit`，Client 检测到 `exit` 后发送 `[exit]` 到 Server 端，即 `["exit"]`。同时，如果想通过直接关闭 Client 端实现退出，也是同样的方式。

时序：设计大致如图：



服务器端 (Server)

首先设置一个 Client 结构体：

```
struct Client {
    SOCKET sock = INVALID_SOCKET;
    string username;

    Client(SOCKET sock = INVALID_SOCKET, string username = "NULL") : sock(sock),
    username(username) {}
};

vector<Client> clients(MAX_CLIENTS);
vector<thread> clientThreads(MAX_CLIENTS);
```

这个结构体实现了定义了一个 Client 所需要的结构，一个 socket 线程以及用户名。

获取时间函数 `GetNowTime`，用来获取本机当前的时间并按照 "%Y-%m-%d %H:%M:%S" 字符串形式输出。

添加头文件<ctime>。

```
string GetNowTime() {
    time_t now = time(nullptr);
    struct tm* ltm = localtime(&now);
    char buffer[20];
    strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", ltm);
    return string(buffer);
}
```

这是线程的处理函数，`index` 为线程索引，用于指定 Client 的处理。

字符数组 `buffer` 用于存储接收到的数据，整型变量 `bytes` 用于存储接收到的字节数。

`recv` 函数从 Client 的 `socket` 接收数据，并存储在 `buffer` 中。如果接收到的字节数大于0，将接收到的数据转换为 C 风格字符串（通过在数据末尾添加空字符 `\0`），并将其赋值给 Client 的 `username` 属性。

`while` 循环使用 `recv` 函数从 Client 接收消息。如果接收到的字节数小于或等于0将跳出循环。否则将接收到的消息转换为字符串，并附加 Client 的用户名，然后通过 `PrintInfo` 和 `ShareMessage` 函数输出并向所有连接这个 Server 的所有 Client 共享这个消息。

当一个 Client 离开聊天室时，将 Client 的 `socket` 标记为无效，并通过 `PrintInfo` 和 `ShareMessage` 输出用户离开的消息。

```
void thread_func(int index) {
    Client& client = clients[index];
    char buffer[BUFFER_SIZE];
    int bytes;

    bytes = recv(client.sock, buffer, BUFFER_SIZE, 0);
    if (bytes > 0) {
        buffer[bytes] = '\0';
        client.username = buffer;

        string welcomeMsg = "welcome " + client.username + " to the chat room!";
        PrintInfo(client.username + " joined the chat room.");
        ShareMessage("System message: " + welcomeMsg);

        while (true) {
            memset(buffer, 0, BUFFER_SIZE);
            bytes = recv(client.sock, buffer, BUFFER_SIZE, 0);
            if (bytes <= 0) break;
            buffer[bytes] = '\0';
            string message = client.username + ": " + buffer;
            PrintInfo("Message: " + message);
            ShareMessage(message);
        }

        client.sock = INVALID_SOCKET;
        string exitMsg = client.username + " has left the chat room.";
        PrintInfo(client.username + " has left the chat room.");
        ShareMessage("System message: " + exitMsg);
    }

    closesocket(client.sock);
}
```

主函数中，Server 端默认使用本机 IP（127.0.0.1），输入端口号后进行 Socket 初始化和启动。

```
// socket初始化
WSADATA wsadata;
WSAStartup(MAKEWORD(2, 2), &wsadata);

SOCKET listen_socket = socket(AF_INET, SOCK_STREAM, 0);
if (-1 == listen_socket){
    cout << "Create listen socket failed. Error Code:" << GetLastError() << endl;
    return -1;
}
```

```

}
else
    cout << "Create listen socket succeeded." << endl;

// 绑定端口号
struct sockaddr_in local = { 0 };
local.sin_family = AF_INET;
local.sin_port = htons(port); // 大小端转换
local.sin_addr.s_addr = inet_addr("0.0.0.0"); // 定义全0地址，全部接受

if (-1 == bind(listen_socket, (struct sockaddr*)&local, sizeof(local))) {
    cout << "Bind listen socket failed. Error Code:" << GetLastError() << endl;
    return -1;
}
else
    cout << "Bind listen socket succeeded." << endl;

// 开启监听
if (-1 == listen(listen_socket, MAX_CLIENTS)) {
    cout << "Start listen failed. Error Code:" << GetLastError() << endl;
    return -1;
}
else
    cout << "Start listen succeeded." << endl;

PrintInfo("Server started on port " + to_string(port));

```

主函数中 `try` 和 `catch` 用于捕获程序异常，发生错误时输出错误信息。

`while` 循环中，`accept` 函数监听 `listen_socket`，等待 Client 的连接请求。`accept` 函数返回一个新创建的 `SOCKET` 对象 `client_socket`，用于与连接的 Client 进行通信。如果 `accept` 函数成功，它将返回一个有效的 `socket`，反之返回 `INVALID_SOCKET`。

如果 `client_socket` 连接有效，调用 `FindFreeThread` 函数来查找空闲的位置为新的 Client 连接分配线程。如果返回 -1，表示服务器已达到其最大容量，则拒绝连接。

找到空闲位置后将新连接的 `socket` 赋值给 `clients` 数组中对应位置的 Client 对象。创建一个新的线程 `clientThreads[pos]`，传入 `thread_func` 函数和位置索引 `pos` 作为参数。这个线程将处理与 Client 的通信。

调用 `detach` 方法，使得新创建的线程独立运行，不与主线程同步。

```

try {
    while (1) {
        SOCKET client_socket = accept(listen_socket, NULL, NULL);

        if (client_socket != INVALID_SOCKET) {
            int pos = FindFreeThread();
            if (pos == -1) {
                PrintInfo("The server has reached its maximum capacity, the connection has failed.");
                closesocket(client_socket);
                continue;
            }

            clients[pos].sock = client_socket;
            clientThreads[pos] = thread(thread_func, pos);
            clientThreads[pos].detach();
        }
    }
}

```

```

    }
}
}
catch (const exception& e) {
    PrintInfo("Server Error: " + string(e.what()));
}

```

客户端 (Client)

`ReceiveMessages` 的目的是从一个指定的 `SOCKET` 接收消息，并在接收到消息时将其打印到控制台。字符数组 `buffer` 用于存储从 `socket` 接收到的数据。`recv` 函数从 `sock` 接收数据，并存储在 `buffer` 中，返回实际接收到的字节数。`bytesReceived` 大于0时，将 `buffer` 中的内容打印到控制台。

```

void ReceiveMessages(SOCKET sock) {
    char buffer[1024];
    while (true) {
        memset(buffer, 0, sizeof(buffer));
        int bytesReceived = recv(sock, buffer, sizeof(buffer), 0);
        if (bytesReceived <= 0) {
            cout << "Disconnected with server." << endl;
            break;
        }
        cout << buffer << endl;
    }
}

```

主函数中前面和 Server 端一样，对 `Socket` 进行初始化操作并绑定端口号。

`getline` 用于读取用户输入，如果用户输入的字符串长度大于0（即用户输入了非空字符串），调用 `send` 函数通过 `client_socket` 发送用户输入的数据到 Server。这段代码会持续读取用户的输入，并将非空的输入发送到服务器，直到用户输入 `exit`。如果发送失败，它会打印一条错误消息并继续循环，让用户有机会重试。

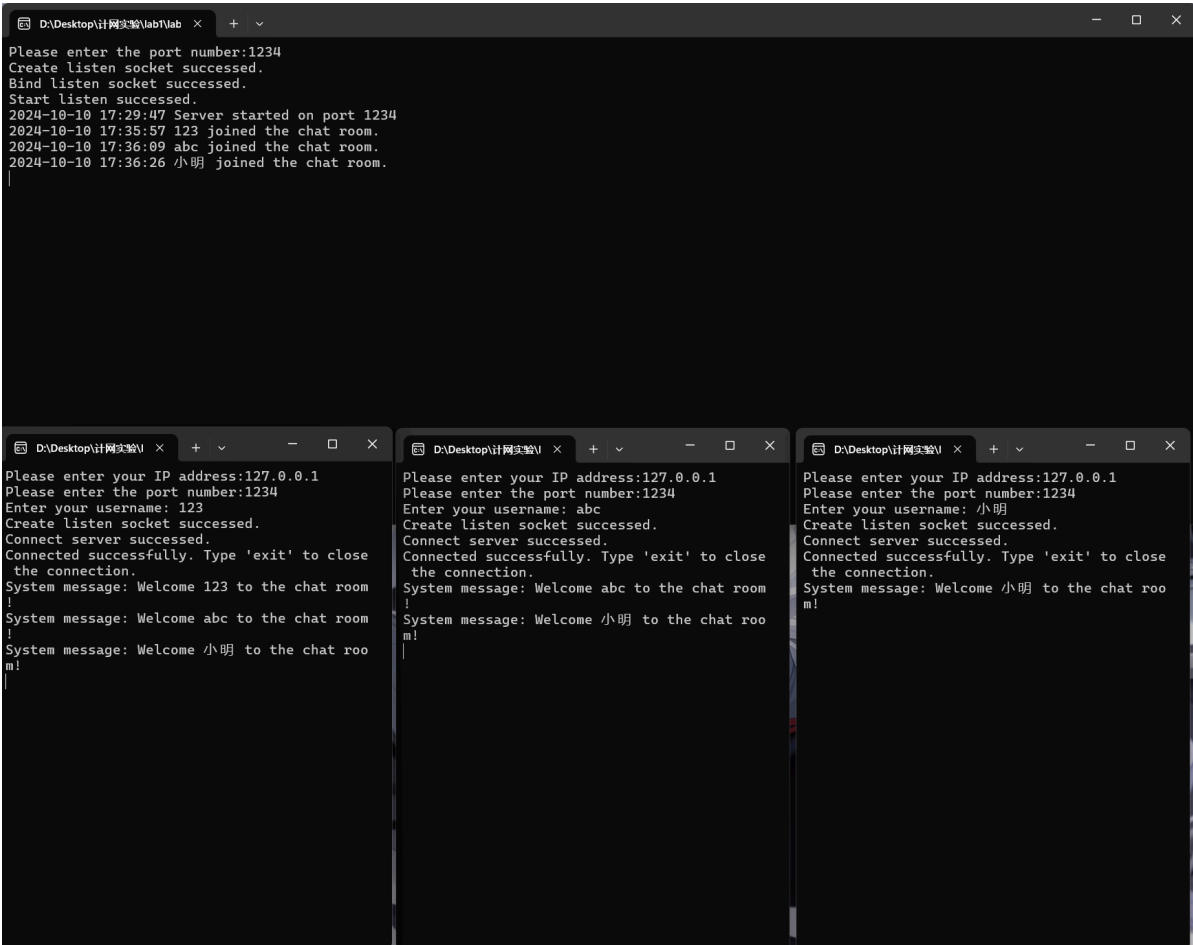
```

do {
    getline(cin, userInput);
    if (userInput.size() > 0) {
        int sendResult = send(client_socket, userInput.c_str(), userInput.size()
+ 1, 0);
        if (sendResult == SOCKET_ERROR) {
            cout << "Send failed. Please try later." << endl;
            continue;
        }
    }
} while (userInput != "exit");

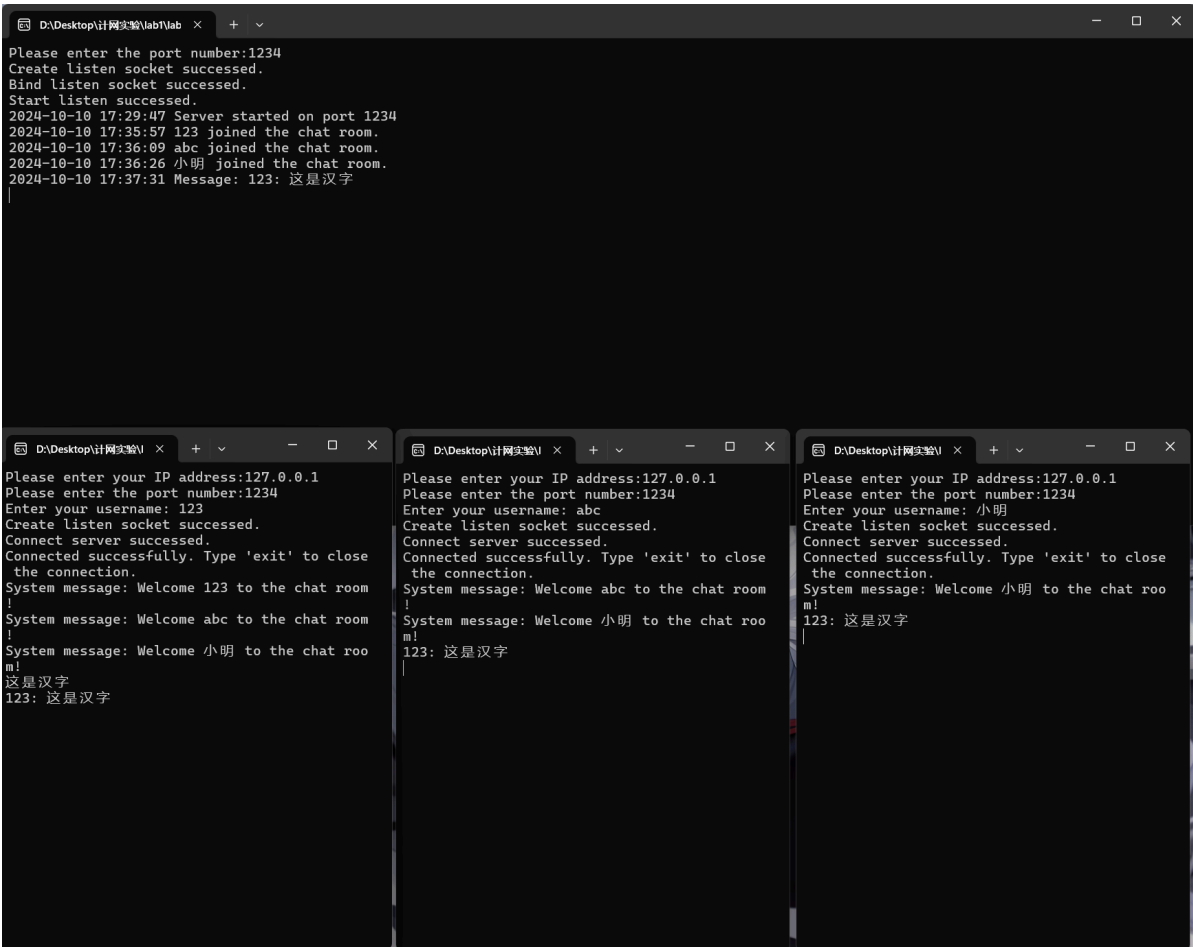
```

效果实现

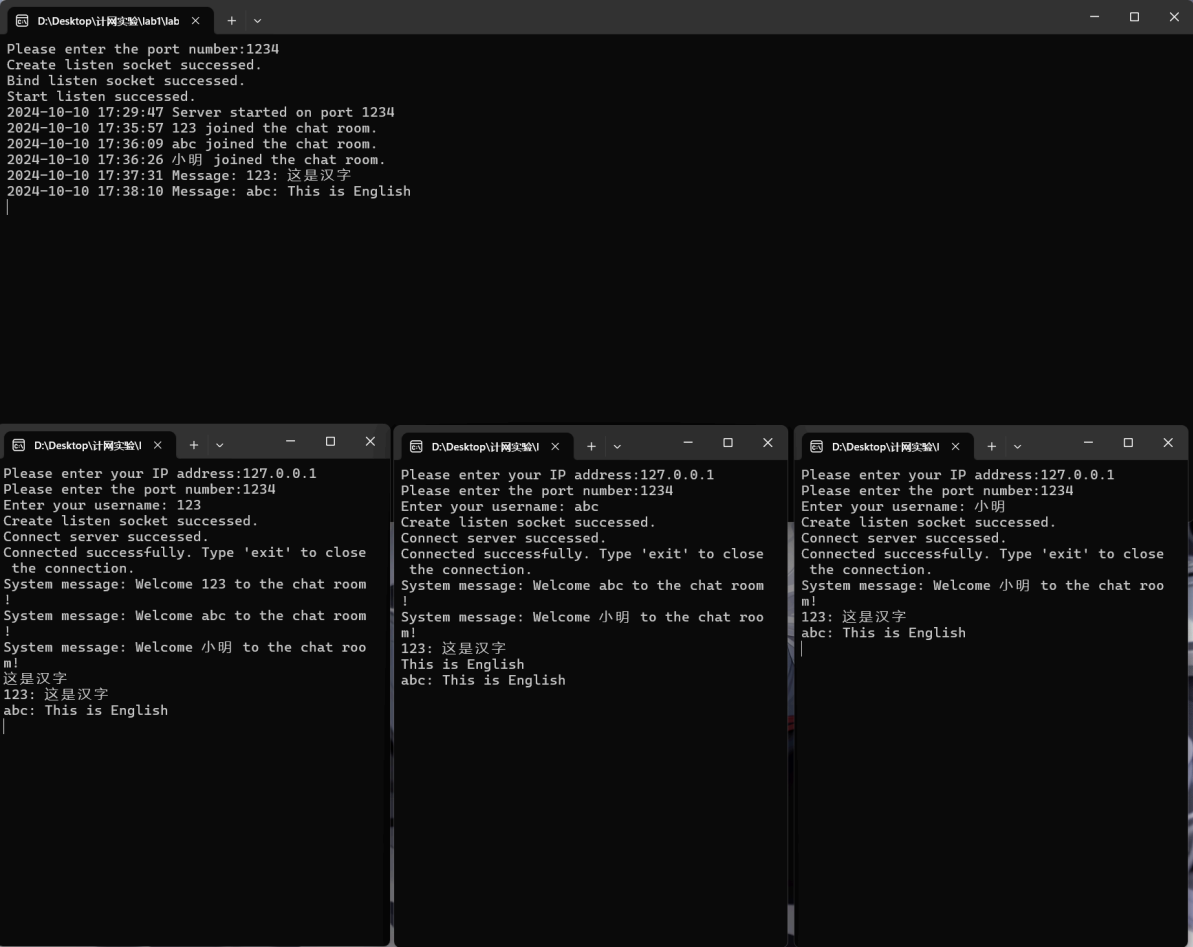
Client 连接 Server



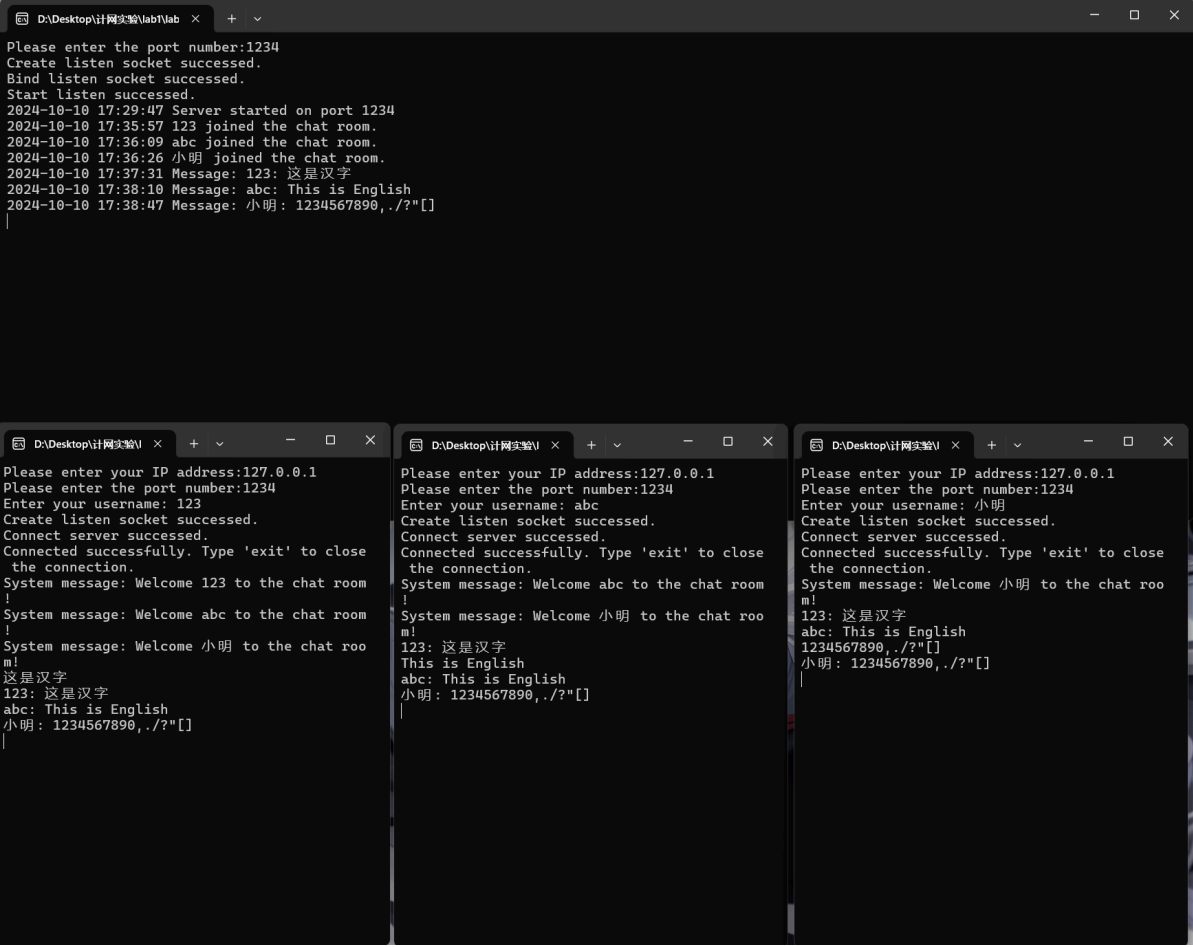
发送消息（汉字）



发送消息（英文）



发送消息（数字和符号）



小明 退出连接

```
D:\Desktop\计网实验\lab\lab x + v
Please enter the port number:1234
Create listen socket succeeded.
Bind listen socket succeeded.
Start listen succeeded.
2024-10-10 17:29:47 Server started on port 1234
2024-10-10 17:35:57 123 joined the chat room.
2024-10-10 17:36:09 abc joined the chat room.
2024-10-10 17:36:26 小明 joined the chat room.
2024-10-10 17:37:31 Message: 123: 这是汉字
2024-10-10 17:38:10 Message: abc: This is English
2024-10-10 17:38:47 Message: 小明: 1234567890,./?["]
2024-10-10 17:39:21 Message: 小明: exit
2024-10-10 17:39:21 小明 has left the chat room.
|

D:\Desktop\计网实验\lab\lab x + v
Please enter your IP address:127.0.0.1
Please enter the port number:1234
Enter your username: 123
Create listen socket succeeded.
Connect server succeeded.
Connected successfully. Type 'exit' to close the connection.
System message: Welcome 123 to the chat room
!
System message: Welcome abc to the chat room
!
System message: Welcome 小明 to the chat room
!
这是汉字
123: 这是汉字
abc: This is English
小明: 1234567890,./?["]
小明: exit
System message: 小明 has left the chat room.
|

D:\Desktop\计网实验\lab\lab x + v
Please enter your IP address:127.0.0.1
Please enter the port number:1234
Enter your username: abc
Create listen socket succeeded.
Connect server succeeded.
Connected successfully. Type 'exit' to close the connection.
System message: Welcome abc to the chat room
!
System message: Welcome 小明 to the chat room
!
123: 这是汉字
This is English
abc: This is English
小明: 1234567890,./?["]
小明: exit
System message: 小明 has left the chat room.
|

D:\Desktop\计网实验\lab\lab x + v
Please enter your IP address:127.0.0.1
Please enter the port number:1234
Enter your username: 小明
Create listen socket succeeded.
Connect server succeeded.
Connected successfully. Type 'exit' to close the connection.
System message: Welcome 小明 to the chat room
!
123: 这是汉字
abc: This is English
1234567890,./?["]
小明: 1234567890,./?["]
exit
小明: exit
|
```

小明 退出连接后发送消息

```
D:\Desktop\计网实验\lab\lab x + v
Please enter the port number:1234
Create listen socket succeeded.
Bind listen socket succeeded.
Start listen succeeded.
2024-10-10 17:29:47 Server started on port 1234
2024-10-10 17:35:57 123 joined the chat room.
2024-10-10 17:36:09 abc joined the chat room.
2024-10-10 17:36:26 小明 joined the chat room.
2024-10-10 17:37:31 Message: 123: 这是汉字
2024-10-10 17:38:10 Message: abc: This is English
2024-10-10 17:38:47 Message: 小明: 1234567890,./?["]
2024-10-10 17:39:21 Message: 小明: exit
2024-10-10 17:39:21 小明 has left the chat room.
2024-10-10 17:40:07 Message: 123: 泥嚎
|

D:\Desktop\计网实验\lab\lab x + v
Please enter your IP address:127.0.0.1
Please enter the port number:1234
Enter your username: 123
Create listen socket succeeded.
Connect server succeeded.
Connected successfully. Type 'exit' to close the connection.
System message: Welcome 123 to the chat room
!
System message: Welcome abc to the chat room
!
System message: Welcome 小明 to the chat room
!
这是汉字
123: 这是汉字
abc: This is English
小明: 1234567890,./?["]
小明: exit
System message: 小明 has left the chat room.
泥嚎
123: 泥嚎
|

D:\Desktop\计网实验\lab\lab x + v
Please enter your IP address:127.0.0.1
Please enter the port number:1234
Enter your username: abc
Create listen socket succeeded.
Connect server succeeded.
Connected successfully. Type 'exit' to close the connection.
System message: Welcome abc to the chat room
!
System message: Welcome 小明 to the chat room
!
123: 这是汉字
This is English
abc: This is English
小明: 1234567890,./?["]
小明: exit
System message: 小明 has left the chat room.
123: 泥嚎
|

D:\Desktop\计网实验\lab\lab x + v
Please enter your IP address:127.0.0.1
Please enter the port number:1234
Enter your username: 小明
Create listen socket succeeded.
Connect server succeeded.
Connected successfully. Type 'exit' to close the connection.
System message: Welcome 小明 to the chat room
!
123: 这是汉字
abc: This is English
1234567890,./?["]
小明: 1234567890,./?["]
exit
小明: exit
|
```

实验反思

在设计过程中根据网络协议三要素：语法、语义及时序，设计了该程序使用的网络协议。

本实验基本完成了实验要求，利用 c++ 语言，使用流式 Socket 实现了多人聊天室的功能。在实现过程中没有使用封装好的 CSocket 类。程序有基本的对话界面，并没有实现图形化界面，这也是本实验的不足之处。Client 端有正常的退出方式：输入 `exit` 退出聊天室。该程序支持中文、英文、数字以及符号的信息发送与接收。

本程序只是一个初级的多人聊天室功能，它的安全性显然非常弱。在实际的聊天软件中，无论是协议还是线程设计都会比这个程序复杂得多。

同时在程序中设置 `buffer[1024]` 也限制了消息发送的最大长度，在超出这个长度之后可能会发生程序崩溃的情况。所以在实验中 Server 的主函数中加入了 `try` 和 `catch` 来处理突发异常情况。

还有一个问题是没有处理用户名的唯一性，在常用的聊天软件中（例如微信、QQ）并没有限制昵称的唯一性，是因为他们有微信号以及QQ号作为区分。在本程序中缺乏区分不同用户的手段，没有处理用户名的唯一性便是这个问题。